

智能体实训@2026

大模型应用实训安排 - 2 周版

项目	安排
实训周期	2026-08-20 至 2026-09-02，共 14 天。
上交时间	2026-09-03 23:59 前。
班级阶段	大四开学前集中实训。
交付形式	每题一个报告，合并为一个 PDF。必做 4 章；第 5 题为选作，可形成第 5 章。
群名	智能体实训@2026

群公告建议稿

群名：智能体实训@2026。实训时间为 2026-08-20 至 2026-09-02，9 月 3 日 23:59 前提交合并 PDF。必做四章：

1. 国内大模型 API 调用实验教程；2. prompt/context/harness/loop agent 演化逻辑；3. pi-core 或类似 agent harness 源码工具调用分析；4. DeepSeek Harness 远程统一研发环境和插件开发。第 5 章为选作：用 DSH 实现一个个人 idea 的 alpha 版。刘昱成、张平竣牵头第 1 章实验教程，要求同学可照着一步步复现。所有截图必须遮挡 API key，代码不得提交明文密钥，参考资料必须给链接和访问日期。

实训背景

这次实训不是单纯学习某一个聊天模型，而是让学生从“大模型接口调用”逐步走到“智能体工程”。大模型应用的基础层是 HTTP API、JSON、鉴权、模型选择、错误处理和费用控制；往上一层是提示词、上下文、工具调用、状态管理、代码执行环境、权限、安全和可观测性；再往上才是能够持续工作、调用工具、检查结果并循环改进的 agent。

近两年的工程趋势很清楚：模型能力越来越强，但可靠应用并不只由模型决定。一个能完成研发任务的智能体，通常需要模型、上下文、工具、执行沙箱、会话日志、任务循环、权限策略、审计和人类介入机制共同构成。DeepSeek Harness 的官方表述是“Agent = Model + Harness”，并把模型、工具、会话、sandbox、loop、调度和 UI 都设计为插件。Pi 则提供了较轻量的 agent harness，可以从源码中直接看到工具调用如何进入循环。

因此，本实训按“API 基础 - 概念演化 - 源码分析 - DSH 应用 - 自选 alpha”的路线推进。学生既要会调国内模型 API，也要能解释为什么 prompt engineering 会扩展为 context engineering、harness engineering 和 loop agent；既要能复现教程，也要能读懂一个工具调用链路并做出自己的小型插件或 alpha 应用。

核心学习目标

- 掌握火山方舟、Kimi、Qwen 等国内大模型 API 的 API key 申请、环境变量配置、HTTP 请求、JSON 请求体、响应解析、流式输出、错误处理和费用记录。
- 能用一个具体任务说明 prompt engineering、context engineering、harness engineering、loop agent 之间的递进关系。
- 能以 Pi 或 DSH 为例画出工具调用链路：模型输出 tool call，运行时校验参数，执行工具，写入 tool result，再进入下一轮模型请求。
- 能远程运行 DSH Web UI，理解 profile、bundle、patch、Cordis plugin 和 tool plugin 的基本概念，并完成至少一个最小工具插件。
- 能把所有过程整理成可复现、可审查、可评分的一份 PDF 报告。

关键概念速览

概念	工程对象	主要解决的问题	学生必须能回答
Prompt engineering	一条或一组指令、示例、输出格式约束。	让单次模型调用更稳定、更符合格式。	怎样把模糊任务改写为可执行指令？怎样通过 few-shot、角色、约束和 JSON schema 控制输出？
Context engineering	模型本次采样能看到的全部 token：系统提示、历史、检索材料、工具说明、文件、摘要、记忆等。	在有限上下文内放入“刚好需要”的信息，避免缺信息或上下文污染。	哪些信息应该进入上下文？如何做裁剪、摘要、引用来源和版本管理？
Harness engineering	模型外部的运行时：工具、权限、沙箱、日志、测试、hooks、CI、知识库、任务计划和观测。	让模型能安全地感知环境、采取行动、被约束、被验证。	工具和权限怎样设计？如何让 agent 看到项目真实状态而不是凭空猜？
Loop agent	反复执行“观察 - 思考/规划 - 调工具 - 观察结果 - 判断是否继续”的控制循环。	处理多步、长时间、不确定、需要反馈修正的任务。	停止条件是什么？失败后如何恢复？谁判断目标是否真的完成？
Agent 本质	模型 + 上下文 + 工具空间 + 状态 + 控制循环 + 评价/停止机制。	把语言模型从回答器变成能在环境中行动的系统。	为什么单个 API 调用不是 agent？为什么只有 while loop 也不够？

两周日程安排

建议按 2026-08-20 开始执行，2026-09-02 完成 PDF 定稿，2026-09-03 提交。如果实际开工日期晚于 8 月 20 日，可压缩阅读时间，但不要压缩第 1 章复现实验和第 4 章 DSH 插件验证。

日期	主题	当天任务	当天产出
8/20 周四	启动与环境	建群；确认分工；安装 Python 3.10+、Node.js、git、curl、jq 或 Apifox；建立报告目录和代码仓库；阅读官方 API 文档。	群公告发布；环境检查截图；个人任务表。
8/21 周五	HTTP、JSON、API key 基础	学习 REST、POST、Header、Bearer token、JSON object/array；用 curl 调通至少一个模型；记录 400/401/429 的含义。	curl 请求与响应截图；API key 脱敏说明；错误码笔记。
8/22 周六	国内模型 API 对比	分别准备火山方舟、Kimi、Qwen 至少两个可用 provider；用统一 Python 脚本切换 base_url/model；测试普通输出、stream、JSON 输出。	统一调用脚本；模型对比表；刘昱成、张平竣收集教程素材。
8/23 周日	第 1 章教程成稿	刘昱成、张平竣把 API key、环境变量、curl、Python、JSON、stream、错误处理写成一步步实验教程；另找一名同学按教程盲测。	第 1 章初稿；盲测问题清单；可复现命令。
8/24 周一	Prompt engineering	阅读 Prompt Report 或 Prompting Guide；用同一个任务做 zero-shot、few-shot、角色、格式约束、JSON schema 对比。	提示词版本表；成功/失败样例；第 2 章框架。
8/25 周二	Context engineering	阅读 Anthropic/Fowler 上下文工程材料；做一个小实验：给模型不同项目文件、规则、历史摘要，比较输出质量和错误。	上下文包设计表；token/信息来源记录；context rot 风险说明。
8/26 周三	Harness 与 loop agent	阅读 ReAct、Anthropic agents、OpenAI harness engineering；写出“prompt -> context -> harness -> loop”的递进逻辑；设计一个最小 agent loop 伪代码。	第 2 章初稿；agent 本质图；伪代码。
8/27 周四	Pi 源码入口	clone Pi；确认`packages/agent`对应包名`@earendil-works/pi-agent-core`；阅读`agent-loop.ts`、`types.ts`、`pi-ai`消息和工具类型。	Pi 架构图；关键文件列表；函数调用链草图。
8/28 周五	工具调用源码深挖	分析 Pi 工具执行：toolCall 过滤、参数校验、before/after hook、并行/串行执行、toolResult 入上下文；对比 DSH tool pipeline。	第 3 章初稿；Pi vs DSH 对比表；1 张时序图。
8/29 周六	DSH 安装与远程使用	本地或远程 Linux 准备 Node 22.19+；运行`npx @deepseek-ai/dsh web`；通过 SSH 隧道访问 3080；配置模型和 workspace。	远程访问截图；安全配置记录；第 4 章环境部分。
8/30 周日	DSH 插件开发	跟做官方 Your first plugin 与 Build a tool；创建`scratch-plugin`、`cordis.yml`、`greet`工具；理解`apply(ctx)`、`inject`、`ctx.tools.register`。	最小插件源码；工具调用截图；问题记录。
8/31 周一	DSH 应用教程复现	选择一个 DSH 应用开发教程跟做。推荐官方插件教程为主，视频教程为辅；把最小工具改成一个业务工具，例如研发环境快照或报告检查。	第 4 章初稿；业务工具 demo；选作 alpha 方案。
9/1 周二	整合与选作	整理四章报告；选作同学完成 alpha 功能、README、demo 截图；统一术语、参考文献、图表编号。	合并 PDF 草稿；选作 demo 初版。
9/2 周三	终审与提交准备	逐项检查复现性、截图脱敏、链接有效、页码、目录、图表、代码片段；同伴互审，修正明显错误。	最终 PDF；提交清单；必要代码附件或仓库链接。
9/3 周四	截止提交	23:59 前提交。迟交需说明原因；明文泄露 API key、严重抄袭、无法复现实验会直接影响成绩。	提交文件：`智能体实训 @2026_姓名_学号.pdf`。

建议分工

- 总协调 1 人：维护日程、检查每日产出、合并 PDF。
- 第 1 章牵头：刘昱成、张平竣。负责把 API 调用教程写到可盲测复现的程度。
- 第 2 章牵头 1-2 人：负责概念演化、文献阅读和同一任务的四层对比实验。

- 第 3 章牵头 1-2 人：负责 Pi/DSH 源码阅读、时序图和设计评价。
- 第 4 章牵头 1-2 人：负责 DSH 远程环境、插件开发和教程复现。
- 编辑与质检 1 人：负责格式、截图脱敏、引用链接、目录、页码、术语统一。

报告结构与评分

提交结构

- 封面：课程/实训名、群名、姓名、学号、分工、提交日期。
- 目录：四章必做；完成选作则加入第五章。
- 每章固定结构：背景说明、实验或阅读目标、步骤与环境、结果截图或源码证据、分析与反思、参考资料。
- 附录：代码仓库地址、环境版本、参考资料总表、截图脱敏声明。
- PDF 建议 25-60 页。截图要压缩但保持可读，代码只放关键片段，完整代码放仓库或附件。

评分标准

模块	分值	评分重点
第 1 章：API 基础实验教程	25	教程可复现；覆盖 API key、HTTP、JSON、curl、Python/Node、stream、JSON 输出、错误处理、费用记录；至少两个国内 provider 调通；截图脱敏。
第 2 章：提示词到 agent 的演化	25	能用自己的实验说明 prompt/context/harness/loop 的递进逻辑；能回答 agent 的本质；有概念图、对比表、失败案例和引用。
第 3 章：pi-core 或类似源码分析	25	能指出关键文件、类和函数；画出工具调用链；分析参数校验、并发、hook、错误路径、会话状态；与 DSH/CodeBuddy/Qoder 至少做一个横向比较。
第 4 章：DSH 融入应用	25	能远程运行 DSH；能说明统一研发环境；跟做官方插件教程；完成一个最小工具插件；有运行截图、源码片段和安全说明。
第 5 章：个人 alpha 选作	+15	有明确用户问题、可运行 alpha、DSH 插件或配置证据、demo 截图/视频、局限性和下一步计划。
扣分或一票否决	-	明文 API key、伪造实验结果、无法复现实验、无引用大段搬运、截图和代码不对应、严重迟交。

统一验收清单

- 所有 API key 均使用环境变量或本地凭据文件，PDF 和仓库中没有明文 key。
- 每个实验都写明系统版本、Node/Python 版本、依赖版本、模型名或 endpoint id、运行日期。
- 所有命令可以复制运行；如果平台变化导致命令失效，报告中说明当时文档版本和替代路径。
- 每章至少有一张图或表：流程图、对比表、源码调用链、远程架构图、插件结构图均可。
- 参考资料给出标题、URL、访问日期；视频资料只作辅助，关键事实以官方文档或源码为准。

第一章：国内大模型 API 基础实验教程

背景

大模型 API 本质上是一个带鉴权的 HTTP 服务。客户端通过 POST 请求发送 JSON 数据，Header 中带 `Content-Type: application/json` 和 `Authorization: Bearer <API\_KEY>`，Body 中通常包含 `model`、`messages`、`stream`、`temperature`、`response\_format` 或 `tools` 等字段。响应也是 JSON，常见字段包括 `choices`、`message`、`usage`、`finish\_reason` 和错误信息。学生首先要把这些基础打牢，否则后面的 agent、工具调用、DSH 插件都会变成黑箱。

国内平台大多提供 OpenAI-compatible 接口，迁移成本较低：同一段 OpenAI SDK 代码通常只需要替换 API key、base_url 和 model。但不同平台在地域、模型名、endpoint id、工具调用、JSON mode、stream usage、错误码和限流策略上仍有差异，报告必须记录这些差异。

必须完成的实验

序号	实验	要求
1	API key 与环境变量	分别申请或使用老师提供的火山方舟、Kimi、Qwen Key。统一写入环境变量：`ARK_API_KEY`、`MOONSHOT_API_KEY`、`DASHSCOPE_API_KEY`。截图必须遮挡 key。
2	curl 调用	至少完成两个 provider 的 `POST /chat/completions` 调用，解释 Header、Body、HTTP 状态码、响应 JSON。
3	Python 或 Node 统一调用	写一个脚本，通过 provider 名称切换 `base_url`、`model`、`api_key`，同一问题同时调用两个模型并输出响应和 usage。
4	流式输出	启用 `stream=True` 或等价参数，记录 chunk 的结构，说明流式输出和普通输出的区别。
5	JSON 输出	让模型输出一个固定 JSON 对象，例如课程计划字段：`title`、`tasks`、`deadline`、`risks`。用 Python `json.loads` 校验。
6	错误处理	故意触发至少两个错误：错误 key、错误模型名、错误 base_url、限流或请求体缺字段。记录错误码和排查过程。
7	模型对比	同一个中文任务分别调用至少两个模型，比较准确性、格式遵循、速度、费用、上下文长度和错误恢复。

统一调用脚本骨架

下面是教程可采用的最小骨架。火山方舟的 `model` 可能是模型 ID，也可能是控制台创建的 Endpoint ID；Qwen 的 `base\_url` 建议以阿里云百炼控制台和官方文档为准。

```
from openai import OpenAI
import json, os, sys

PROVIDERS = {
    "kimi": {
        "env": "MOONSHOT_API_KEY",
        "base url": "https://api.moonshot.cn/v1",
        "model": "kimi-k3",
    },
    "qwen": {
        "env": "DASHSCOPE_API_KEY",
        "base url": "https://dashscope.aliyuncs.com/compatible-mode/v1",
        "model": "qwen-plus",
    },
    "ark": {
        "env": "ARK_API_KEY",
        "base url": "https://ark.cn-beijing.volces.com/api/v3",
        "model": "■■■■■■ Model ID ■ Endpoint ID",
    },
}

def call(provider: str, prompt: str):
```

```

cfq = PROVIDERS[provider]
client = OpenAI(api key=os.environ[cfq["env"]], base url=cfq["base url"])
resp = client.chat.completions.create(
    model=cfq["model"],
    messages=[
        {"role": "system", "content": "■■■■■■■■■■■■■■■■■■■■"},
        {"role": "user", "content": prompt},
    ],
)
print(resp.choices[0].message.content)
print(ison.dumps(resp.usage.model_dump(), ensure_ascii=False))

if name == " main ":
    call(sys.argv[1], "■■■■■■ HTTP Header ■ JSON Body ■ API Key ■■■■■")

```

第 1 章报告模板

- 1. 背景：国内大模型 API、OpenAI-compatible、HTTP/JSON/API key 的基本关系。
- 2. 环境：系统、Python/Node 版本、依赖版本、provider、模型名、base_url、运行日期。
- 3. 实验步骤：从申请 key、配置环境变量、curl、Python/Node，到 stream 和 JSON 校验。
- 4. 结果：至少两个 provider 的请求、响应、usage、错误处理截图。
- 5. 对比分析：模型响应质量、格式遵循、延迟、费用、限流、文档易用性。
- 6. 可复现教程：让没有经验的同学能按步骤完成第一次调用。

第二章：提示词到智能体的演化

背景

prompt engineering 解决的是“如何问”的问题，适合单次任务和短链路任务。随着任务变长，模型需要看到项目文件、约束、历史、工具说明、测试结果和外部数据，于是问题变成“模型应该看到什么”，这就是 context engineering。再往后，模型不仅要看，还要行动：读文件、写文件、跑命令、搜索、调用 API、开子任务，这需要 harness engineering。最后，多步任务必须不断观察结果、修正计划、判断是否结束，这才进入 loop agent。

演化的内在逻辑可以概括为：任务复杂度越高、时间跨度越长、外部环境越真实、错误代价越大，工程重心就越从“写好一句 prompt”转向“管理上下文、约束工具、记录状态、验证结果和设计循环”。

建议实验：同一任务四层递进

层级	实验方式	观察重点
Prompt	只给一句任务：`写一个学生 API 调用教程`。	是否遗漏前置条件、API key 安全、错误处理、复现步骤？
Context	加入官方 API 文档摘要、示例代码、学生基础、报告模板。	输出是否更具体？是否引用来源？是否仍有过时或幻觉？
Harness	加入脚本、测试、JSON 校验、截图清单、评分 rubrics。	模型是否能被工具和检查项约束？是否减少主观空话？
Loop agent	让 agent 按计划写教程、运行脚本、读错误、修改、再次验证，直到清单通过。	循环是否真的改正错误？停止条件是否可靠？token 和时间成本如何？

必须回答的问题

- prompt engineering、context engineering、harness engineering、loop agent 分别控制什么？
- 为什么 context engineering 不是简单把资料全部塞进 prompt？
- 为什么工具说明也是 prompt/context 的一部分？
- harness 与 framework、IDE、MCP、plugin、sandbox、CI 的关系是什么？
- agent 的本质是什么？请给出你自己的定义，并用一次工具调用循环证明。

Agent 的一个务实定义

定义

Agent 是一个由模型驱动、在外部 harness 约束下运行的闭环系统。它维护任务状态，把合适上下文交给模型，允许模型选择工具或动作，把观察结果写回状态，并根据目标、检查器或人工反馈决定继续、修正、求助或停止。单次 LLM API 调用不是 agent；只有模型没有工具和状态也不是完整 agent；只有 while loop 但没有目标检查、权限和可观测性也不可靠。

第 2 章报告模板

- 1. 概念背景：分别定义四个 engineering 层级，说明它们不是替代关系，而是外层包住内层。
- 2. 演化逻辑：用“信息不足、任务变长、需要行动、需要反馈验证”解释为什么会演化。
- 3. 实验对比：同一个任务做四层递进，保留 prompt、上下文包、工具/检查器、循环日志。
- 4. Agent 本质：给出定义、图示和至少一个工具调用循环例子。
- 5. 反思：哪些任务不需要 agent？哪些任务用 agent 风险更大？如何设计停止条件？

第三章：Pi-core 源码与工具调用分析

背景

源码分析的目标不是通读每一行，而是抓住 agent harness 的关键链路：模型怎样收到工具 schema，怎样输出 tool call，运行时怎样校验和执行，工具结果怎样进入会话历史，循环怎样决定继续或停止。Pi 是较适合教学的例子：仓库目录目前是 `packages/agent`，但包名是 `@earendil-works/pi-agent-core`；它的 `agent-loop.ts` 可以直接看到工具调用的执行路径。

Pi 必读路径

文件或目录	阅读重点
`packages/agent/package.json`	确认包名 `@earendil-works/pi-agent-core`、依赖和定位。
`packages/agent/src/types.ts`	`AgentLoopConfig`、`AgentToolCall`、`AgentToolResult`、before/after hook、tool execution mode。
`packages/agent/src/agent-loop.ts`	主循环：stream assistant response、提取 toolCall、串行/并行执行、创建 toolResult、进入下一轮。
`packages/ai/src/types.ts`	底层 Message、AssistantMessage、ToolCall、ToolResultMessage 的数据结构。
`packages/ai/src/api/*`	不同 provider 如何把统一消息转换为 OpenAI、Anthropic、Google、Mistral 等协议。
`packages/coding-agent/src/core/tools/*`	内置 read/write/edit/bash/grep/find 等工具的定义和执行。
`packages/coding-agent/src/core/tools/tool-definition-wrapper.ts`	ToolDefinition 与 AgentTool 的包装关系。
`packages/coding-agent/src/core/extensions/wrapper.ts`	扩展工具如何包装进 AgentTool，并把新增工具名反馈给运行时。
`packages/coding-agent/src/core/agent-session.ts`	工具注册、启用、系统提示重建、tool event 映射到会话 UI。

Pi 工具调用链路

```
user message
-> runAgentLoop / runLoop
-> streamAssistantResponse
-> assistant message contains content blocks
-> filter blocks where type == "toolCall"
-> validateToolArguments(tool, toolCall)
-> beforeToolCall hook may block
-> execute tool sequentially or in parallel
-> afterToolCall hook may replace result
-> create ToolResultMessage
-> append to context.messages
-> next LLM request sees tool result
```

DSH 对比路径

文件或文档	阅读重点
`docs/architecture.md`	Cordis、profiles/bundles、core packages、turn flow、session log、capability seams。
`docs/tool-execution-pipeline.md`	`tool/call`、`tools/pre-execute`、guards、`tools/execute`、`tools/post-execute`、`tools/result`、`tool/result`。
`packages/core/tools/src/index.ts`	`ToolRuntime`、`ctx.tools.register`、`defineTool`、tool pipeline、Code Mode presentation。

文件或文档	阅读重点
<code>`packages/core/agent-loop/src/agent.ts`</code>	ReactLoopAgent : turn、step、pre-step、build request、assistant message、tool calls。
<code>`packages/core/agent-loop/src/tool-calls.ts`</code>	工具调用调度：并行/独占、按模型顺序提交结果、session append。
<code>`docs/user/develop/basic/index.md`</code>	第一个插件： <code>`apply(ctx)`</code> 、 <code>`cordis.yml`</code> overlay、自动清理和 <code>`inject`</code> 。
<code>`docs/user/develop/basic/tool.md`</code>	最小工具插件： <code>`ctx.tools.register(defineTool(...))`</code> 。

源码报告必须包含

- 一张“模型请求 - 工具调用 - 工具结果 - 下一轮请求”的时序图。
- 至少 8 个关键文件路径，说明每个文件在工具调用链路中的职责。
- 至少 3 个关键函数或类的解释，例如 ``runLoop``、``executeToolCalls``、``prepareToolCall``、``ToolRuntime.execute``、``ReactLoopAgent.step``。
- 至少 4 条错误路径：未知工具、参数校验失败、工具抛异常、请求被中止、输出 token 截断、权限拒绝任选其四。
- 至少 2 个设计评价：例如并行工具调用、hook 位置、session log、工具 schema、上下文压缩、权限/沙箱边界。
- 与 CodeBuddy、Qoder、DSH 或 OpenCode 至少做一个横向比较。闭源产品可以基于官方 SDK/文档比较能力，不要编造源码细节。

第四章：DeepSeek Harness 融入具体应用

背景

DeepSeek Harness (dsh) 是 DeepSeek 官方开源的 agent harness，目前处于 developer preview。它的核心设计是“Everything is a plugin”：模型、工具、skills、sessions、sandboxes、storage、loops、scheduling 和 UI 都由插件组合。它基于 Cordis，插件通过`apply(ctx)`往上下文注册能力，通过 profile、bundle 和 patch 组合出不同运行时。

本章要求学生不仅能启动 DSH，还要把它放进真实研发流程中：远端统一环境、SSH 隧道访问、模型配置、workspace 选择、会话记录、插件开发和安全约束。注意：DSH 仍在快速迭代，报告必须记录版本、commit 或 npm 包版本，避免把当前行为写成永久事实。

远程统一研发环境建议方案

步骤	命令或说明
1. 远端机器	Ubuntu 或其他 Linux/macOS 均可；不要求 GPU。建议 4C8G 起步，磁盘 30GB+。
2. 基础依赖	`git`、`curl`、Node.js 22.19+ 或 24+、Corepack/pnpm。DSH 源码仓库当前 package.json 要求`node: ^22.19.0    >=24.0.0`。
3. 目录规划	`~/agent-lab/projects` 放项目；`~/dsh-agent-lab/` 放每个学生的 DSH_HOME； `~/agent-lab/secrets.env` 保存本地环境变量并`chmod 600`。
4. 启动 DSH	快速体验：`npx @deepseek-ai/dsh web`。源码方式：`git clone`、`pnpm install`、`pnpm run build`、 `pnpm dsh web`。
5. 安全访问	不要把 3080 暴露到公网。远端监听 loopback，本地执行：`ssh -L 3080:127.0.0.1:3080 user@server`，然后浏览器打开`http://127.0.0.1:3080`。
6. 模型配置	Web UI: Settings -> Models，填 DeepSeek 或自定义 OpenAI-compatible provider。Key 写入 DSH 凭据或环境变量，不写进仓库。
7. Workspace	选择远端项目目录作为 workspace。新会话先让 agent 总结仓库结构，再进行修改类任务。
8. 日志与复现	记录启动命令、端口、Node 版本、DSH 版本、模型 provider、workspace 路径、关键截图和一次完整会话。

推荐跟做教程

首选教程

我建议把 DSH 官方开发文档作为主教程：`Your first plugin` + `Build a tool`。原因是它最贴近源码当前状态，直接覆盖`scratch-plugin`、`cordis.yml`、`apply(ctx)`、`inject`、`ctx.tools.register(defineTool(...))`，并且能从最小插件自然扩展到业务工具。视频资料可作为演示辅助，尤其适合解决安装和 UI 操作问题，但关键命令以官方文档为准。

必须完成的 DSH 插件实验

- 跟做官方`Your first plugin`：创建`scratch-plugin/src/my-plugin.ts`，写`apply(ctx)`，用`cordis.yml` overlay 加载，看到终端输出。
- 跟做官方`Build a tool`：注册`greet`工具，让模型调用并返回`Hello, Ada!`。
- 把`greet`改造成一个业务工具，例如`dev\_environment\_snapshot`：返回当前目录、git 分支、Node/Python 版本、package manager、关键文件是否存在。
- 解释`inject = ['tools']`为什么必要，`defineTool`的`parameters`、`output.schema`、`render`、`execute`分别负责什么。
- 至少设计一个安全约束：只读工具优先；执行命令必须白名单；敏感文件不读；API key 不进入工具返回内容；高风险命令要求人工确认。

第 4 章报告模板

- 1. DSH 背景：developer preview、Everything is a plugin、Cordis、profile/bundle/patch。
- 2. 远程环境：机器、Node/pnpm、DSH 启动方式、SSH 隧道、安全设置、模型配置、workspace。
- 3. 官方教程复现：截图、命令、插件文件、`cordis.yml`、运行结果。
- 4. 业务插件：需求、工具 schema、源码片段、调用截图、安全边界。
- 5. 原理分析：插件如何挂载，工具 schema 如何进入模型上下文，tool result 如何回到下一步。
- 6. 问题与改进：版本变化、兼容性、权限、远程访问、团队共享、后续 alpha 方向。

第五章选作：用 DSH 实现个人 Alpha

第 5 题为选作，但强烈建议有能力的同学完成。Alpha 不要求功能完整，但必须能运行、能演示、能说明为什么适合用 DSH 或插件化 agent harness 来做。

可选方向

方向	Alpha 说明
实训报告助教	检查 PDF 结构、API key 脱敏、引用完整性、实验截图是否齐全，给出修改建议。
统一研发环境巡检	读取项目依赖、git 状态、测试脚本、README、AGENTS.md，生成可执行的 onboarding 报告。
API 费用与错误日志分析	读取调用日志，统计 provider、模型、token、费用、错误码，给出降本建议。
课程资料问答插件	把课程公告、参考链接、评分标准作为上下文，回答同学关于提交和实验的问题。
代码变更安全检查	对 agent 修改过的 diff 做规则检查：是否动了敏感文件、是否缺测试、是否泄露 key。
个人效率工具	例如论文阅读摘要、Bilibili/YouTube 教程笔记、项目日报自动生成，但必须体现 DSH tool/plugin。

Alpha 交付标准

- 一句话问题定义：用户是谁，痛点是什么，为什么聊天机器人不够。
- 最小架构图：模型、上下文、工具、插件、会话、远程环境、输入输出。
- 可运行证据：启动命令、插件源码、demo 截图或短视频、至少一次真实输入输出。
- 安全与边界：能读什么、能写什么、哪些操作需要人工确认、如何避免 key 泄露。
- 局限性：当前 alpha 不能做什么，下一版优先改什么。

辅助资料使用建议

不要把参考资料理解成“越多越好”。每一章应先读官方文档或源码，再用论文建立概念框架，最后用视频和社区文章补足安装演示。报告引用时要写清资料用途：是验证命令、解释概念、对比设计，还是复现实验。

分章资料路线

章节	优先资料	使用方式
第 1 章 API 基础	MDN HTTP、HTTP methods、headers、status codes；RFC 8259 JSON；Kimi、Qwen、火山方舟、DeepSeek API 官方文档；curl、jq、Postman/Apifox。	用官方文档确认 base_url、endpoint、鉴权头、model 字段；用 MDN/RFC 解释 HTTP 与 JSON；用 curl 和 Python SDK 做同一请求的复现。
第 2 章概念演化	OpenAI/Anthropic/Google/Microsoft prompt 文档；Anthropic 与 LangChain context engineering；ReAct、LLM agent survey、12-Factor Agents。	把资料按 prompt -> context -> harness -> loop agent 排列，说明为什么任务复杂度上升后需要从“写提示词”走向“设计运行时”。
第 3 章源码分析	Pi GitHub、Pi 官网、Pi agent types；DSH architecture、tool pipeline、core tools、agent-loop 源码；Qoder 与 CodeBuddy SDK 文档。	源码只抓工具调用主链路：工具 schema 如何进入模型、tool call 如何被解析校验、执行结果如何回写上下文、hook/权限/错误如何处理。
第 4 章 DSH 应用	DSH 官方页、GitHub、Quickstart、Configure models、Your first plugin、Build a tool、Cordis paper；VS Code Remote SSH、OpenSSH port forwarding。	先本地启动，再远程启动并用 SSH 隧道访问 3080；插件开发以官方教程为主，社区视频只用于补充界面操作。
第 5 章选作 Alpha	DeepSeekPlugins 目录、awesome-deepseek-harness、DSHarness101、OpenAI/Martin Fowler harness engineering、Harness Handbook。	选一个真实小痛点，把模型、上下文、工具、插件、会话日志和验证方式画成最小架构图，不要求完整产品化。

资料筛选规则

- 官方文档和源码优先。凡是 API 参数、安装命令、版本要求、权限模型，必须以官方文档或仓库 README 为准。
- 论文和长文用于解释框架。不要把论文摘要直接贴进报告，要转化为你自己的任务图、流程图和对比表。
- 视频用于复现路径。Bilibili/YouTube 视频可以帮助跟做，但报告里必须说明对应的官方命令或源码位置。
- 社区教程要做版本标记。DSH 仍处于 developer preview，报告中要写清访问日期、npm 包版本或 Git commit。
- 所有截图必须脱敏。API key、账户余额、内网 IP、服务器用户名、真实项目密钥都不得进入提交 PDF。

最低引用要求

报告	最低引用
第 1 章	至少 3 个国内模型官方 API 文档 + 2 个 HTTP/JSON/curl 基础资料。
第 2 章	至少 2 个厂商 prompt/context 官方文档 + 2 篇 agent/harness 论文或深度文章。
第 3 章	至少 1 个源码仓库 + 3 个具体源码文件路径 + 1 个类似工具文档作横向比较。
第 4 章	至少 4 个 DSH 官方文档/源码链接 + 1 个远程开发资料 + 1 个视频教程复现记录。
第 5 章选作	至少 2 个同类应用或插件资料 + 自己 alpha 的源码/截图/运行日志。

参考资料

以下资料已扩充为分层清单。官方文档、源码和论文优先级最高；视频适合入门演示，但必须结合官方文档核对命令。
访问日期建议统一写为 2026-08-17。

类别	资料	链接
HTTP/JSON	MDN Overview of HTTP	打开链接
HTTP/JSON	MDN HTTP request methods	打开链接
HTTP/JSON	MDN HTTP headers	打开链接
HTTP/JSON	MDN HTTP status codes	打开链接
HTTP/JSON	RFC 8259 JSON	打开链接
HTTP/JSON	ECMA-404 JSON PDF	打开链接
HTTP/JSON	JSON.org	打开链接
HTTP/JSON	curl official site	打开链接
HTTP/JSON	jq manual	打开链接
HTTP/JSON	Python Requests Quickstart	打开链接
HTTP/JSON	Postman quick start	打开链接
HTTP/JSON	Apifox API testing beginner guide	打开链接
国内 API	火山方舟快速入门	打开链接
国内 API	火山方舟 Chat API	打开链接
国内 API	火山方舟兼容 OpenAI SDK	打开链接
国内 API	火山方舟 Base URL 与鉴权	打开链接
国内 API	火山方舟接入三方工具	打开链接
国内 API	Kimi API 快速开始	打开链接
国内 API	Kimi API Overview	打开链接
国内 API	Kimi OpenAI compatibility	打开链接
国内 API	Kimi K2.6 quickstart	打开链接
国内 API	阿里云百炼 Qwen OpenAI-compatible	打开链接
国内 API	阿里云百炼首次调用千问 API	打开链接
国内 API	阿里云百炼文本生成 API 参考	打开链接
国内 API	阿里云百炼 DashScope API 参考	打开链接
国内 API	DeepSeek API first call	打开链接
国内 API	DeepSeek models and pricing	打开链接
国内 API	DeepSeek Responses API	打开链接
Prompt/Context	OpenAI Prompt engineering	打开链接
Prompt/Context	OpenAI prompt best practices help	打开链接
Prompt/Context	Anthropic prompt engineering overview	打开链接
Prompt/Context	Anthropic prompting best practices	打开链接
Prompt/Context	Google Gemini prompt design strategies	打开链接
Prompt/Context	Microsoft Foundry prompt engineering	打开链接

类别	资料	链接
Prompt/Context	Microsoft system message design	打开链接
Prompt/Context	Prompt Engineering Guide	打开链接
Prompt/Context	Prompting Guide context engineering	打开链接
Prompt/Context	Anthropic prompt interactive tutorial	打开链接
Prompt/Context	The Prompt Report PDF	打开链接
Prompt/Context	A Systematic Survey of Prompt Engineering in LLMs	打开链接
Prompt/Context	Anthropic: Effective context engineering for AI agents	打开链接
Prompt/Context	Sourcegraph context engineering guide	打开链接
Prompt/Context	LangChain context engineering docs	打开链接
Prompt/Context	LangChain context engineering blog	打开链接
Prompt/Context	Martin Fowler: Context Engineering for Coding Agents	打开链接
Prompt/Context	Context Engineering from prompts to architecture	打开链接
Prompt/Context	Context engineering for OSS agents	打开链接
Agent/Harness	Anthropic: Building Effective AI Agents	打开链接
Agent/Harness	LangChain agents docs	打开链接
Agent/Harness	Lilian Weng LLM powered agents	打开链接
Agent/Harness	12-Factor Agents	打开链接
Agent/Harness	ReAct PDF	打开链接
Agent/Harness	ReAct project page	打开链接
Agent/Harness	LLM autonomous agents survey	打开链接
Agent/Harness	LLM agent planning survey	打开链接
Agent/Harness	LLM agent evaluation survey	打开链接
Agent/Harness	Tool use evolution survey	打开链接
Agent/Harness	Agent memory mechanisms survey PDF	打开链接
Agent/Harness	Awesome Function Callings	打开链接
Agent/Harness	ToolACE paper PDF	打开链接
Agent/Harness	OpenAI: Harness engineering	打开链接
Agent/Harness	Martin Fowler: Harness engineering for coding agent users	打开链接
Agent/Harness	AI Harness Engineering	打开链接
Agent/Harness	Code as Agent Harness	打开链接
Agent/Harness	Harness Handbook	打开链接
Agent/Harness	Security controls through harness engineering	打开链接
Pi	Pi GitHub	打开链接
Pi	Pi project website	打开链接
Pi	Pi core types docs	打开链接
Pi	Pi coding agent README	打开链接
DSH	DeepSeek Harness 官方页	打开链接

类别	资料	链接
DSH	DeepSeek Harness GitHub	打开链接
DSH	DSH docs directory	打开链接
DSH	DSH Web UI Quickstart	打开链接
DSH	DSH Web UI guide	打开链接
DSH	DSH Configure models	打开链接
DSH	DSH Architecture	打开链接
DSH	DSH Tool execution pipeline	打开链接
DSH	DSH Your first plugin	打开链接
DSH	DSH Build a tool	打开链接
DSH	Cordis paper PDF	打开链接
DSH 社区	awesome-deepseek-harness	打开链接
DSH 社区	DeepSeekPlugins directory	打开链接
DSH 社区	DeepSeekPlugins docs	打开链接
DSH 社区	DSHarness101	打开链接
DSH 社区	OpenClaw DSH guide	打开链接
DSH 社区	MindStudio setup guide	打开链接
DSH 社区	MindStudio DSH explained	打开链接
DSH 社区	The New Stack DSH plugins article	打开链接
远程开发	VS Code Remote SSH docs	打开链接
远程开发	VS Code SSH tutorial	打开链接
远程开发	OpenSSH manual pages	打开链接
远程开发	GitHub SSH key generation	打开链接
远程开发	GitHub add SSH key	打开链接
远程开发	Node.js downloads	打开链接
相关工具	Qoder docs	打开链接
相关工具	Qoder Agent SDK Quick Start	打开链接
相关工具	CodeBuddy Agent SDK	打开链接
相关工具	Tencent Cloud CodeBuddy overview	打开链接
相关工具	Bruno API client	打开链接
YouTube	老师给定 DSH 原理视频	打开链接
YouTube	DeepSeek Harness - Its a Big Deal!	打开链接
YouTube	DeepSeek Harness + V4 Flash	打开链接
YouTube	DeepSeek Harness Setup	打开链接
YouTube	Deepseek Harness: Everything is a plugin	打开链接
YouTube	DeepSeek Harness: The End of Claude Code?	打开链接
YouTube	Effective Context Engineering for AI Agents	打开链接
YouTube	Context Engineering in 29 Minutes	打开链接

类别	资料	链接
YouTube	Advanced Context Engineering for Agents	打开链接
YouTube	Context Engineering with LangChain and Manus	打开链接
YouTube	Context engineering explained	打开链接
YouTube	Building AI Agent Harnesses playlist	打开链接
Bilibili	【极速入门】DeepSeek Harness 上手实战指南	打开链接
Bilibili	DeepSeek Harness 上手流程+入门导读	打开链接
Bilibili	deepseek harness 简单介绍	打开链接
Bilibili	DeepSeek Harness 可视化技术解读	打开链接
Bilibili	Harness Engineering 深度解析与最佳实践	打开链接
Bilibili	Bilibili Harness 搜索结果	打开链接
Bilibili	Bilibili DeepSeek 搜索结果	打开链接

版本提醒
DSH、Pi、Qoder、CodeBuddy、国内模型 API 都在快速迭代。报告中必须记录访问日期、npm 包版本或 Git commit。如果教程命令与当前版本不一致，以官方文档和源码为准，并在报告中说明差异。